

CI/CD with Jenkins

CI/CD automates the process of taking a code change from a developer's machine to a tested, packaged, and potentially deployed application.

The central idea is simple:

If a repeatable software-delivery process follows predictable steps, a machine should perform those steps consistently.

1. The Problem Before CI/CD

Imagine a developer changes a file in a Node.js application and pushes the code to GitHub. Before the application can reach users, someone must perform several tasks:

1. Download the latest source code.
2. Install the required dependencies.
3. Compile or build the application.
4. Run automated tests.
5. Package the application.
6. Copy it to a server or deployment environment.
7. Configure and start the application.
8. Verify that it is working correctly.

For a Node.js project, some of the commands might be:

```
git pull
npm ci
npm test
npm run build
```

For a Spring Boot project:

```
git pull
mvn test
mvn clean package
```

These steps can be performed manually for a small application maintained by one developer. The process becomes unreliable, however, when several developers are continuously changing the same codebase.

Common mistakes include:

- Forgetting to run the tests
- Deploying the wrong branch
- Copying an old JAR file
- Building with an inconsistent dependency version
- Deploying something that works locally but fails on the server

The underlying problem is not that people are careless. The problem is that a repetitive and predictable delivery process is being performed manually.

2. The Fundamental Idea of CI/CD

Without automation, the workflow looks like this:

```
Developer
  ↓
Get the latest code
  ↓
Install dependencies
  ↓
Build
  ↓
Test
  ↓
Package
  ↓
Deploy
```

CI/CD moves this responsibility to an automated system:

```
Developer pushes code
↓
Automation starts
↓
Build → Test → Package → Deploy
```

This automated sequence is called a **pipeline**.

A pipeline is a predefined sequence of automated steps through which source code moves toward becoming a releasable or deployed application.

Jenkins and GitHub Actions are examples of tools that can execute these pipelines.

Why is it called “continuous”?

Continuous does not mean that an application must be deployed every second. It means that integration, testing, packaging, and release preparation happen regularly as part of normal development.

The older release model often looked like this:

```
Develop for several months
↓
Perform one large integration
↓
Run one large testing cycle
↓
Release everything together
```

CI/CD encourages smaller feedback cycles:

```
Small change → Integrate → Verify
Small change → Integrate → Verify
Small change → Integrate → Verify
```

Small changes are easier to understand, test, review, and fix.

3. Continuous Integration

Continuous Integration (CI) is the practice of frequently integrating code changes into a shared codebase and automatically verifying those changes.

CI has two essential parts:

Frequent integration + Automated verification

A simple CI example

Suppose an application contains this function:

```
function add(a, b) {  
  return a + b;  
}
```

After the developer pushes the change, Jenkins can execute:

```
GitHub push  
↓  
Jenkins pipeline  
↓  
Checkout source code  
↓  
npm ci  
↓  
npm test
```

If the tests pass, the build can continue.

Now suppose the function is accidentally changed to:

```
function add(a, b) {  
  return a - b;  
}
```

The next push triggers the same pipeline. An automated test might report:

```
Test failed  
Expected: 5
```

Received: -1

CI has now detected the defect before the change moves further through the delivery process.

CI is more than unit testing

A CI pipeline may include:

- Dependency installation
- Compilation or application build
- Unit tests
- Integration tests
- Code linting
- Static code analysis
- Security scanning
- Artifact packaging

Not every project needs every check. The purpose of CI is to determine quickly and consistently whether a new change is healthy enough to continue.

If a required check fails, the pipeline should stop. Bad software should not casually move toward production.

4. Build Artifacts

After the code successfully passes its validations, the build usually produces something that can be stored and deployed. This output is called an **artifact**.

An artifact is a versioned output produced by the build process that can be preserved, distributed, or deployed.

Examples include:

- Spring Boot `.jar` files
- Java `.war` files

- `.zip` or `.tgz` packages
- Compiled native binaries
- Docker images

For example:

```
payment-service-2.3.1.jar
```

Artifacts should normally be stored in a central location instead of being rebuilt differently on every server. Common artifact-storage systems include Nexus, JFrog Artifactory, and container registries.

5. Continuous Delivery vs Continuous Deployment

The abbreviation **CD** can refer to either Continuous Delivery or Continuous Deployment. The difference is the production-release decision.

Practice	What is automated?	Production release
Continuous Integration	Build and verification of integrated changes	Not necessarily included
Continuous Delivery	Build, test, package, and release preparation	Requires a human or business decision
Continuous Deployment	Build, test, package, and deployment	Happens automatically after all checks pass

Continuous Delivery

Continuous Delivery keeps every successful change in a state where it can be released safely at any time.

A typical flow might be:

```
Code
↓
Build
↓
Test
```

```
graph TD; A[ ] --> B[Package]; B --> C[Development]; C --> D[QA]; D --> E[Staging]; E --> F[Manual approval]; F --> G[Production];
```

The pipeline prepares and validates the release, but production deployment remains a human or business decision. A company may release every Friday, wait for a product manager's approval, or coordinate the release with customer communication.

Continuous Deployment

Continuous Deployment automatically deploys every change that passes all required pipeline checks to production.

```
graph TD; A[Developer push] --> B[Build]; B --> C[Tests]; C --> D[Package]; D --> E[Staging validation]; E --> F[Production deployment];
```

There is no mandatory approval button for every release. This model requires strong automated testing, reliable monitoring, and confidence in the complete delivery process.

Continuous Delivery asks, "Can we release this safely?" Continuous Deployment adds, "If it is safe, release it automatically."

6. A Typical Pipeline Lifecycle

A production pipeline commonly follows this lifecycle:

```
Code → Source → Build → Test → Package → Publish → Deploy → Verify
```

6.1 Code

A developer creates a:

- Feature
- Bug fix
- Refactor
- Configuration change

6.2 Source

The code reaches a version-control platform such as GitHub, GitLab, or Bitbucket. This may happen through a direct push or after a pull request is merged.

That source-control event can trigger the pipeline.

6.3 Build

The application is prepared for execution or packaging.

Node.js example:

```
npm ci
npm run build
```

Spring Boot example:

```
mvn clean package
```

6.4 Test

The pipeline validates the application through checks such as:

- Unit tests
- Integration tests
- Linting
- Static analysis
- Security checks

A required failure stops the pipeline.

6.5 Package

The build creates a deployable artifact, such as:

```
app.jar
```

In a container-based workflow, the artifact may be a Docker image.

6.6 Publish

The artifact is placed in permanent, centralized storage such as Nexus, Artifactory, or a container registry.

Publishing a versioned artifact allows the same tested output to move between environments.

6.7 Deploy

The artifact is released into one or more environments:

```
Development → Testing → Staging → Production
```

6.8 Verify

Deployment is not complete merely because the deployment command returned successfully. The pipeline or monitoring system should verify:

- Did the application start?
- Is its health endpoint responding?

- Can users access it?
- Are error rates increasing?
- Are important dependencies available?

The real goal is a healthy application, not just a successful deployment command.

7. What Is Jenkins?

For automation to work, a system must:

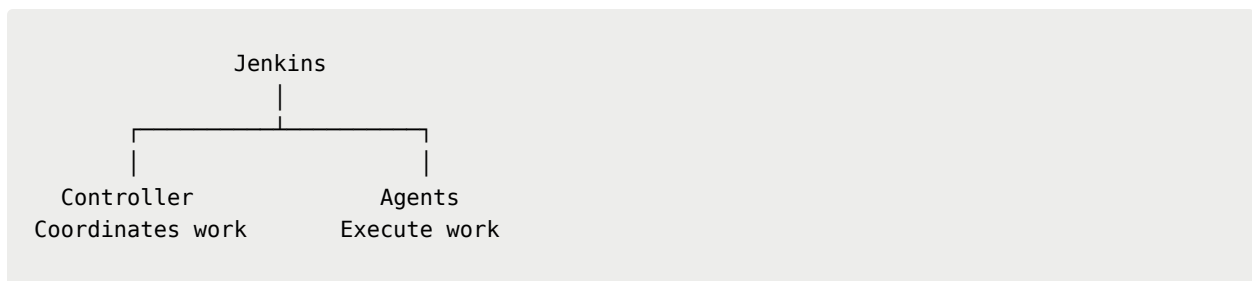
1. Detect or receive a build request.
2. Find a machine on which the work can run.
3. Download the source code.
4. Execute the required commands.
5. Record the result.

Jenkins is an automation server that coordinates and executes jobs and pipelines in response to events or instructions.

It can automate tasks such as building, testing, packaging, and deploying applications.

8. Jenkins Architecture

At a high level, Jenkins uses a controller-agent architecture:



An easy mental model is:

```
Controller = Manager  
Agent      = Worker
```

8.1 Jenkins Controller

The controller manages Jenkins and coordinates automation. Its responsibilities include:

- Jobs and pipeline definitions
- Build queue and scheduling
- Credentials
- Plugins
- Build history
- Agent management
- Jenkins configuration and UI

When a new build arrives, the controller determines where and when it can run.

8.2 Should the controller build applications?

Jenkins can execute work on its built-in node, which is convenient for a learning environment or small demo. In a production setup, build workloads should generally run on separate agents.

A resource-heavy Maven build could consume most of the controller's CPU and memory while the controller is also trying to serve the UI, schedule builds, communicate with agents, and maintain pipeline state.

The preferred separation is:

```
Controller → Coordinates Jenkins  
Agent      → Executes build workloads
```

8.3 Jenkins Agent

An agent is a compute environment available to Jenkins for executing work. It may be a:

- Physical machine
- Virtual machine
- Cloud VM
- Container-based environment
- Kubernetes-provisioned worker

Different agents can support different operating systems or toolchains:

```
Spring Boot build → Linux agent  
.NET build        → Windows agent  
iOS build         → macOS agent
```

Why use multiple agents?

Different environments

Projects may require different operating systems, hardware, dependencies, or tools.

Parallel execution

If three builds each take ten minutes, one worker may need approximately thirty minutes to execute them sequentially. Three available workers may execute them at the same time and finish in approximately ten minutes.

Isolation

One project may require Java 17, another Java 21, and another Node.js 24. Separate agents help provide controlled and predictable environments.

9. Node, Agent, Executor, Queue, and Workspace

These terms describe different parts of the Jenkins execution model.

Term	Meaning
Node	A machine or compute environment Jenkins can use
Agent	A worker connected to the Jenkins controller
Executor	A logical slot that allows Jenkins work to run on a node
Build queue	The waiting area for work when no suitable executor is available
Workspace	The directory in which Jenkins keeps a job's source code and generated files while working

9.1 Node and agent

A **node** is a machine or compute environment known to Jenkins. The controller has a built-in node, while additional worker nodes connect to it as agents.

In everyday discussion, the words *node* and *agent* are sometimes used interchangeably, but the distinction above provides a useful mental model.

9.2 Executor

An executor is a logical slot for running Jenkins work.

Suppose a Linux agent has two executors:

```
Linux agent
├─ Executor 1 → Build A
└─ Executor 2 → Build B
```

The agent may run two Jenkins workloads concurrently. Configuring four executors tells Jenkins that it may schedule up to four applicable tasks on that node at the same time.

Executors are not extra CPU cores or extra memory. The underlying machine must still have enough resources to support the configured concurrency.

9.3 Build queue

If all suitable executors are busy, new work waits in the build queue:

```
Executor 1 → Building App A
Executor 2 → Building App B
```

App C → Waiting in queue

When an executor becomes free, the controller assigns the waiting work to it.

9.4 Workspace

A workspace is the directory Jenkins uses while executing a particular job.

For a Node.js project, it may contain:

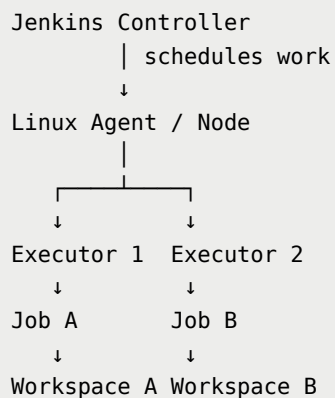
```
package.json
src/
tests/
node_modules/
dist/
```

For a Spring Boot project:

```
pom.xml
src/
target/
```

Pipeline commands normally execute relative to the current workspace.

Complete execution model



Remember it this way:

- **Node:** Where can the work execute?
 - **Executor:** How many Jenkins tasks may run concurrently?
 - **Workspace:** Where are the files used by that work?
-

10. Jenkins Jobs and Pipelines

Infrastructure gives Jenkins a place to execute commands, but Jenkins still needs a definition of what to do.

Job

A Jenkins job is a configured automation task that Jenkins knows how to execute.

For example, a job named `Build Node Application` might specify:

```
Repository: github.com/company/app
```

```
Steps:
```

1. `npm ci`
2. `npm test`
3. `npm run build`

Pipeline as Code

If these steps are configured only through the Jenkins UI, the application code lives in Git but the instructions for building it live only inside the Jenkins server.

This creates several problems:

- Pipeline changes are harder to review.
- The build configuration has no clear version history.
- Recreating Jenkins becomes more difficult.
- Someone can change the process without changing the repository.

Pipeline as Code stores the automation definition with the application:

```
application-repository/  
├─ src/  
├─ tests/  
├─ package.json  
├─ package-lock.json  
└─ Jenkinsfile
```

The build and delivery instructions can now receive the same benefits as application code:

- Version control
- Pull requests
- Code review
- Change history
- Reproducibility

11. Jenkinsfile

A Jenkins pipeline is commonly defined in a file named:

```
Jenkinsfile
```

It normally has no file extension and is placed at the root of the project.

Jenkins Pipeline uses a domain-specific language based on Groovy syntax. Basic pipelines can be written without first learning the full Groovy language.

Declarative vs Scripted Pipeline

Jenkins supports two main pipeline styles:

Style	Characteristics
Declarative Pipeline	Structured, opinionated, readable, and easier to validate and teach
Scripted Pipeline	More flexible and programmatic, but easier to make complex

Declarative Pipeline is generally the better starting point for beginners and for pipelines that do not require highly dynamic logic.

12. Declarative Pipeline Structure

A basic Declarative Pipeline answers four questions:

1. Is this a pipeline definition?
2. Where should it run?
3. What stages does it contain?
4. What steps should execute in each stage?

The structure is:

```
pipeline
  ↓
agent
  ↓
stages
  ↓
stage
  ↓
steps
```

12.1 `pipeline`

```
pipeline {
}
```

The top-level `pipeline` block contains the complete Declarative Pipeline definition.

12.2 `agent`

```
pipeline {
    agent any
}
```

`agent any` tells Jenkins to execute the pipeline on any available eligible agent. Jenkins allocates an executor and workspace on that agent.

12.3 stages

```
pipeline {  
    agent any  
  
    stages {  
    }  
}
```

The `stages` block contains the logical phases of the pipeline.

12.4 stage

```
stages {  
    stage('Build') {  
    }  
  
    stage('Test') {  
    }  
}
```

A stage is a named phase such as:

- Install Dependencies
- Build
- Test
- Security Scan
- Package
- Deploy

Meaningful stages make the result easy to understand:

```
Install   
Test 
```

Build 
Deploy Skipped

12.5 **steps**

```
stage('Test') {  
    steps {  
        sh 'npm test'  
    }  
}
```

The **steps** block contains the actual operations performed during a stage. The **sh** step executes a shell command on a Unix-like agent.

13. Complete Node.js Jenkinsfile

```
pipeline {  
    agent any  
  
    stages {  
        stage('Install') {  
            steps {  
                sh 'npm ci'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                sh 'npm test'  
            }  
        }  
  
        stage('Build') {  
            steps {  
                sh 'npm run build'  
            }  
        }  
    }  
}
```

```
}  
  }  
}  }
```

In plain English, this definition says:

1. Create a pipeline.
2. Use any available eligible agent.
3. Install the exact dependencies described by the lock file.
4. Run the automated tests.
5. Build the application.

The stages run in sequence. If a required command returns a failure, Jenkins marks the stage and build as failed, and later stages normally do not run.

14. What Happens Internally?

When Jenkins runs the pipeline, the process is conceptually:

1. The Jenkins controller receives the pipeline request.
2. Jenkins reads the pipeline definition.
3. The controller finds an eligible agent.
4. Jenkins allocates an executor.
5. Jenkins allocates a workspace.
6. The source code is checked out into that workspace.
7. `npm ci` installs dependencies.
8. `npm test` runs the tests.
9. `npm run build` builds the application.
10. Jenkins records the logs, stage results, and final build status.

The repository contents become available inside the agent's workspace:

GitHub repository		Jenkins workspace
src/	→	src/
tests/	→	tests/
package.json	→	package.json
Jenkinsfile	→	Jenkinsfile

Pipeline commands such as `npm ci` then run inside that workspace.

15. Jenkins Triggers

The pipeline is useful only when Jenkins knows when to run it. A **trigger** starts a build in response to an event or schedule.

A GitHub push can notify Jenkins through a webhook:

```
Developer pushes code
    ↓
GitHub sends a webhook event
    ↓
Jenkins receives the event
    ↓
Jenkins starts the pipeline
```

Other possible triggers include:

- A manual build request
- A scheduled build
- A pull request or branch event
- Another Jenkins job
- An external API request

The GitHub webhook trigger can be configured during the practical Jenkins demo.

16. Final Mental Model

```
Developer pushes code
↓
Git platform sends an event
↓
Jenkins Controller receives the request
↓
Controller selects an Agent/Node
↓
An Executor becomes available
↓
Jenkins prepares a Workspace
↓
Jenkins reads the Jenkinsfile
↓
Build → Test → Package → Deploy → Verify
↓
Jenkins records the result
```

Key takeaways

- CI/CD replaces repetitive manual delivery work with a consistent automated pipeline.
- Continuous Integration frequently combines and automatically verifies changes.
- Continuous Delivery keeps software ready for release but retains a manual production decision.
- Continuous Deployment automatically releases every change that passes the required checks.
- Jenkins coordinates jobs and pipelines through a controller and one or more agents.
- Nodes provide compute, executors provide concurrency slots, and workspaces hold job files.
- A `Jenkinsfile` stores the delivery process with the source code.
- Declarative Pipeline organizes work using `pipeline`, `agent`, `stages`, `stage`, and `steps`.
- A trigger, such as a GitHub webhook, can start the pipeline automatically after a code change.